FlowDesk

Document Approval & Workflow System

SYSTEM DOCUMENTATION

Version 1.0 • March 2026

Prepared by Mukela Katungu

Live Demo: flowdesk-demo.netlify.app

Table of Contents

1. System Overview 3

1.1 Tech Stack 3

1.2 System Stats 3

2. Architecture Diagram 4

3. User Roles & Permissions 5

4. Document Workflow 6

4.1 Upload & Metadata 6

4.2 The Approval Loop 6

4.3 Resubmission Flow 7

5. Database Schema 8

6. API Reference 9

6.1 Authentication 9

6.2 Documents 9

6.3 Webhooks 10

7. Webhook Integration 11

7.1 Inbound Webhooks 11

7.2 Outbound Webhooks 11

8. File Handling & Preview 12

9. Authentication & Security 13

10. Production Adaptation Guide 14

1. System Overview

FlowDesk is a web-based document approval and workflow management system. It manages the lifecycle of accounting and purchase documents through a "Review → Approve/Reject → Archive" pipeline. Documents enter the system via direct web upload or inbound webhooks (Google Forms / n8n), and upon approval, an outbound webhook triggers archiving in external systems like Paperless-ngx.

1.1 Tech Stack

LAYER	TECHNOLOGY	PURPOSE
Frontend	React 18 + Vite + TypeScript	Single-page application with routing
Styling	Tailwind CSS + Custom CSS Variables	Responsive design with dark sidebar theme
Backend	Express.js (Node.js + TypeScript)	REST API, authentication, file handling
Database	PostgreSQL (Supabase)	All application data with UUID primary keys
Auth	JWT (jsonwebtoken + bcryptjs)	Token-based authentication with role enforcement
File Storage	Cloudinary	PDF and image upload, storage, and transformation
Frontend Host	Netlify	Static site hosting with SPA redirects
Backend Host	Railway	Dockerized Node.js server
Icons	Lucide React + LordIcon	UI iconography with animated icons

1.2 System Stats

15	5	7	2
API ENDPOINTS	DB TABLES	PAGES	USER ROLES

2. Architecture Diagram

The system uses a decoupled frontend/backend architecture. The React SPA communicates with the Express API server via REST. The API server coordinates between PostgreSQL (data), Cloudinary (files), and external systems (n8n, Google Forms) via webhooks.

```
+------------------+          +-------------------+
|  Google Forms    |          |   n8n Workflow    |
|  (Field Data)    |          |   (Automation)    |
+---------+---------+          +---------+---------+
          |                              |
     Inbound Webhook              Outbound Webhook
          |                              ^
          v                              |
+---------+-----------------------------+---------+
|                Express API Server               |
|  /api/auth/*     Authentication (JWT + bcrypt)  |
|  /api/documents/*  CRUD + Workflow actions      |
|  /api/webhooks/*   Inbound receive + logs       |
+---------+----------------+----------------+-----+
          |                |                |
  +-------+------+  +-----+------+  +------+------+
  |  PostgreSQL  |  |  Cloudinary |  |   Resend    |
  |  (Supabase)  |  |  (Files)    |  |  (Email)*   |
  |  5 Tables    |  |  PDFs/Imgs  |  |  Optional   |
  +--------------+  +-------------+  +-------------+
          ^
          |  REST API (HTTPS + JWT)
          |
+---------+----------+
|    React SPA       |
|    (Netlify)       |
|  Vite + TypeScript |
+--------------------+

* Email notifications not yet implemented (planned)
```

Data Flow Summary

Input Path A: Web Upload

- Accountant logs in and uploads PDF with metadata
- File stored on Cloudinary, metadata in PostgreSQL
- Document enters "Pending Review" state

Input Path B: Webhook (Google Forms)

- Google Forms submits to `/api/webhooks/inbound`
- System creates document record automatically
- Document enters "Pending Review" with source = "webhook"

3. User Roles & Permissions

The system enforces two distinct roles with different permission levels. Role checks happen at both the API middleware layer (server-side) and the UI layer (conditional rendering).

CAPABILITY	MANAGER	ACCOUNTANT
View all documents	Yes	Yes
View dashboard with stats	Yes (Manager Dashboard)	Yes (Standard Dashboard)
Upload new documents	No	Yes
Approve documents	Yes	No
Reject documents (with note)	Yes	No
Delete documents	Yes	No
Replace & resubmit rejected documents	No	Own documents only
View webhook logs	Yes	Yes
View audit trail	Yes	Yes

Role Enforcement

Server-Side (Middleware)

- `authMiddleware` — validates JWT token on all protected routes
- `requireManager` — blocks non-managers from approve/reject/delete
- `requireAccountant` — blocks non-accountants from upload/resubmit

Client-Side (UI)

- Upload button only shown to accountants
- Approve/Reject buttons only shown to managers
- Resubmit button only shown to the original uploader
- Manager sees "Manager Dashboard" heading

Resubmission Permission Logic

The resubmit action has a compound check: the user must be an accountant AND the document must have status `"needs_correction"` AND the user must be the original uploader (`uploadedBy.id === user.id`). This prevents one accountant from modifying another's rejected document.

4. Document Workflow

This is the core of FlowDesk. Every document moves through a defined lifecycle from upload to archive, with full audit trail at every step.

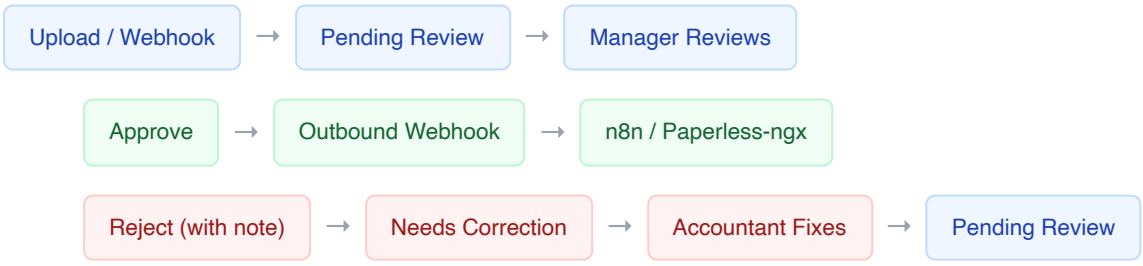
4.1 Upload & Metadata

When an accountant uploads a document, they provide:

FIELD	TYPE	EXAMPLE
Title	Free text	"Q1 Office Supplies Payment"
Document Type	Dropdown	Payment Voucher, Journal Voucher, Purchase Order, Invoice, Receipt, Credit Note
Related Party	Free text	"Staples Inc."
Date	Date picker	2026-03-09
File	PDF, PNG, JPG	Max 10MB

The file is uploaded via `multipart/form-data` to Cloudinary. The Cloudinary URL and file metadata are stored in PostgreSQL. An audit event "Document uploaded" is created automatically.

4.2 The Approval Loop



What happens on Approve:

- Document status changes to `"approved"`
- Manager's name and timestamp stored as `approved_by` and `approved_at`
- Audit event: "Approved by manager"
- Outbound webhook fires to configured archive endpoint (n8n)
- Audit event: "Outbound webhook triggered"
- Webhook log entry created with direction, URL, status, and payload

What happens on Reject:

- Document status changes to `"needs_correction"`
- Rejection note stored on the document
- Audit event: "Rejected by manager" with the rejection note
- Rejection banner appears on the document detail page

4.3 Resubmission Flow

When a document is rejected, the original uploader (accountant) sees a "Replace & Resubmit" button. On click:

- A file picker opens for the corrected document
- The new file replaces the old one on Cloudinary
- Document version number increments (v1 → v2)
- Status resets to `"pending_review"`
- Rejection note is cleared
- Audit event: "Resubmitted (version N)" recorded
- Document re-enters the approval queue for the manager

Version Tracking

Each resubmission increments the document's `version` field. The version number is displayed as a badge in the UI (e.g., "Version 2"). The full history of actions is preserved in the audit trail, so there's a complete record of every upload, rejection, and resubmission.

5. Database Schema

Five tables in PostgreSQL with UUID primary keys. Foreign keys enforce referential integrity. All timestamps are stored with timezone (TIMESTAMPTZ).

users

COLUMN	TYPE	NOTES
`id`	UUID (PK)	Auto-generated
`name`	VARCHAR(255)	Display name
`email`	VARCHAR(255) UNIQUE	Login identifier
`password_hash`	VARCHAR(255)	bcrypt hash (10 rounds)
`role`	VARCHAR(20)	`'manager'` or `'accountant'`
`created_at`	TIMESTAMPTZ	Account creation time

documents

COLUMN	TYPE	NOTES
`id`	UUID (PK)	Auto-generated
`title`	VARCHAR(500)	Document title
`type`	VARCHAR(50)	payment_voucher, journal_voucher, purchase_order, invoice, receipt, credit_note
`status`	VARCHAR(30)	pending_review, needs_correction, approved, archived
`related_party`	VARCHAR(500)	Company or person name
`date`	DATE	Document date
`file_name`	VARCHAR(500)	Original filename
`file_size`	INTEGER	Bytes
`file_path`	VARCHAR(1000)	Cloudinary URL
`source`	VARCHAR(20)	`'web_upload'` or `'webhook'`
`version`	INTEGER	Starts at 1, increments on resubmit
`rejection_note`	TEXT	Manager's rejection reason
`uploaded_by`	UUID (FK)	References users
`approved_by`	UUID (FK)	References users (NULL until approved)
`approved_at`	TIMESTAMPTZ	Approval timestamp
`webhook_triggered`	BOOLEAN	Whether outbound webhook was sent

document_events (Audit Trail)

COLUMN	TYPE	NOTES
`id`	UUID (PK)	Auto-generated
`document_id`	UUID (FK)	References documents (CASCADE delete)
`action`	VARCHAR(500)	e.g. "Document uploaded", "Approved by manager"
`user_name`	VARCHAR(255)	Who performed the action
`note`	TEXT	Optional (used for rejection notes)
`created_at`	TIMESTAMPTZ	When the event occurred

webhook_logs & webhook_config

TABLE	PURPOSE	KEY COLUMNS
`webhook_logs`	Records every webhook (inbound/outbound)	direction, url, method, status, payload, document_id
`webhook_config`	Stores configured webhook endpoints	name, url, direction, enabled

6. API Reference

All API endpoints are prefixed with `/api`. Authentication is via Bearer JWT token in the `Authorization` header. Protected endpoints return 401 if the token is missing or invalid.

6.1 Authentication

METHOD	ENDPOINT	AUTH	PURPOSE
POST	`/api/auth/signup`	Public	Create new account (name, email, password, role)
POST	`/api/auth/login`	Public	Login, returns JWT token + user object
GET	`/api/auth/me`	Required	Validate token, return current user

6.2 Documents

METHOD	ENDPOINT	AUTH	PURPOSE
GET	`/api/documents`	Required	List all documents (supports ?status, ?type, ?search filters)
GET	`/api/documents/stats`	Required	Dashboard stats (counts by status)
GET	`/api/documents/:id`	Required	Document detail with audit trail and uploader/approver info
POST	`/api/documents`	Accountant	Upload document (multipart/form-data with file)
POST	`/api/documents/:id/approve`	Manager	Approve document, triggers outbound webhook
POST	`/api/documents/:id/reject`	Manager	Reject document with note (JSON body: {note})
POST	`/api/documents/:id/resubmit`	Accountant	Replace file, bump version, return to pending
DELETE	`/api/documents/:id`	Manager	Permanently delete document

6.3 Webhooks

METHOD	ENDPOINT	AUTH	PURPOSE
POST	`/api/webhooks/inbound`	Public*	Receive documents from Google Forms / n8n
GET	`/api/webhooks/logs`	Required	List all webhook activity (inbound + outbound)
GET	`/api/webhooks/config`	Required	Get configured webhook endpoints

***Inbound Webhook Security**

The inbound webhook endpoint is currently public to accept Google Forms submissions. For production, it should be secured with an API key or HMAC signature verification to prevent unauthorized document injection.

Response Formats

All responses are JSON. Successful operations return the relevant data. Errors return:

```
{
  "error": "Human-readable error message"
}

Status codes:
  200  Success
  201  Created (signup, upload)
  400  Bad request (missing fields)
  401  Unauthorized (invalid/missing token)
  403  Forbidden (wrong role)
  404  Not found
  409  Conflict (duplicate email)
  500  Internal server error
```

7. Webhook Integration

FlowDesk supports both inbound and outbound webhooks, enabling integration with Google Forms, n8n, and archive systems like Paperless-ngx.

7.1 Inbound Webhooks (Google Forms → FlowDesk)

External systems can submit documents to FlowDesk by POSTing to `/api/webhooks/inbound`:

```
POST /api/webhooks/inbound
Content-Type: application/json

{
  "title": "Field Purchase - Site Materials",
  "type": "purchase_order",
  "relatedParty": "BuildRight Supplies",
  "date": "2026-03-09",
  "source": "Google Forms"
}

Response: 200 { "id": "uuid", "message": "Document created from webhook" }
```

The document is created with status `"pending_review"` and source `"webhook"`. A webhook icon appears in the UI to distinguish webhook-sourced documents from web uploads.

7.2 Outbound Webhooks (FlowDesk → n8n / Paperless-ngx)

When a manager approves a document, FlowDesk automatically sends an outbound POST to the configured archive endpoint:

```
POST https://n8n.example.com/webhook/archive
Content-Type: application/json

{
  "documentId": "a980b3ce-d901-4db9-92ab-081d52d7dab8",
  "title": "Information Security Assignment",
  "type": "invoice",
  "status": "approved",
  "archiveTo": "paperless-ngx"
}

The webhook URL is configured in the webhook_config table.
The response status (200, 500, etc.) is logged in webhook_logs.
```

Webhook Monitoring

Every webhook (inbound and outbound) is logged in the `webhook_logs` table with direction, URL, HTTP method, response status, full payload, and timestamp. The Webhook Logs page in the UI provides a real-time view of all webhook activity.

8. File Handling & Preview

Upload Pipeline

Browser selects file → FormData POST to API → Multer + Cloudinary Storage → URL stored in PostgreSQL

- **Max file size:** 10MB
- **Allowed formats:** PDF, PNG, JPG, JPEG
- **Storage:** Cloudinary `flowdesk/` folder with `resource_type: auto`
- **URL format:** `https://res.cloudinary.com/{cloud}/image/upload/v{ver}/flowdesk/{id}.pdf`

Document Preview

PDFs are previewed inline using Cloudinary's image transformation API. The first page of each PDF is converted to a high-resolution JPG on the fly:

```
Original:  .../upload/v1773069114/flowdesk/abc123.pdf
Preview:   .../upload/pg_1,w_800/v1773069114/flowdesk/abc123.jpg

The transformation pg_1 extracts page 1, w_800 sets width to 800px.
Cloudinary generates and caches the JPG automatically.
Images (PNG, JPG) are displayed directly without transformation.
```

A "Download" link provides access to the original file. The preview renders as an `<img>` tag, ensuring cross-browser compatibility without requiring a PDF viewer plugin.

9. Authentication & Security

JWT Authentication

User submits email + password → Server verifies with bcrypt → JWT token generated (24h expiry) → Token stored in localStorage

The JWT payload contains: `{ id, role, name, iat, exp }`. Every API request includes the token in the `Authorization: Bearer <token>` header. On 401, the frontend auto-redirects to login.

Security Measures

- **Password hashing:** bcrypt with 10 salt rounds (passwords never stored in plaintext)
- **Role-based middleware:** `requireManager` and `requireAccountant` enforce role checks server-side
- **CORS:** Configured to allow only the production frontend domain and localhost for development
- **File validation:** Multer enforces 10MB limit and allowed file extensions
- **SQL injection prevention:** All database queries use parameterized queries (`$1, $2` placeholders)
- **Token expiry:** JWT tokens expire after 24 hours, requiring re-login
- **Auto-redirect on 401:** Expired or invalid tokens trigger automatic redirect to login page

Session Recovery

On page load, the app checks for an existing token in localStorage and validates it by calling `GET /api/auth/me`. If the token is expired or invalid, the user is silently redirected to login. This provides persistent sessions across browser refreshes without storing sensitive data.

10. Production Adaptation Guide

The current demo is deployed on cloud services (Supabase, Cloudinary, Railway, Netlify). Below is what needs to change for the production deployment described in the project spec: a self-hosted Docker setup on a Linux server with PostgreSQL 17+.

What's Already Complete

REQUIREMENT	STATUS	NOTES
Manager role (approve/reject/delete)	Done	Full permissions with audit trail
Accountant role (upload/resubmit)	Done	Restricted to own rejected documents
PDF upload with metadata	Done	Date, Type (dropdown), Related Party
Approval loop with rejection notes	Done	Status transitions + audit events
Outbound webhook on approval	Done	Fires POST to configured URL
Inbound webhook endpoint	Done	Accepts JSON from Google Forms / n8n
Secure login (JWT + bcrypt)	Done	Role-based access control
PostgreSQL database	Done	5 tables, UUID PKs, full schema
Full audit trail	Done	Every action logged with timestamp + user
Webhook logs monitoring	Done	Inbound + outbound with payload
Source code provided	Done	Full TypeScript codebase

What Needs to Change for Production

ITEM	CURRENT (DEMO)	PRODUCTION CHANGE	EFFORT
Docker setup	Railway (PaaS)	Dockerfile + docker-compose.yml with Node.js + PostgreSQL 17 containers	~2 hours
Database	Supabase (hosted)	Self-hosted PostgreSQL 17+ in Docker, same schema	~30 min
File storage	Cloudinary (cloud)	Local Docker volume or S3-compatible storage (MinIO)	~2 hours
Outbound webhook URL	n8n.example.com	Point to real n8n instance URL	Config change
Inbound webhook auth	Public endpoint	Add API key or HMAC verification	~1 hour
Email notifications	Not implemented	Add email on rejection/approval (Nodemailer or similar)	~2 hours
HTTPS/SSL	Handled by Netlify/Railway	Nginx reverse proxy with Let's Encrypt	~1 hour

Docker Compose Structure (Planned)

The production docker-compose.yml will include three services: `app` (Node.js Express server serving both API and React frontend), `db` (PostgreSQL 17), and `nginx` (reverse proxy with SSL). The frontend will be built and served statically by the Express server. A single `docker compose up` command will start everything.

Database Migration

The migration script (`server/src/db/migrate.ts`) automatically creates all tables with `CREATE TABLE IF NOT EXISTS` . It runs on server startup, making the database setup fully automatic. No manual SQL is needed.